LEVEL: BLACK
CLEARANCE: 0x00

THE SOVEREIGN AI STACK

```
// sovereignty mode: ENABLED
// cloud dependency: FALSE
// telemetry: DISABLED
// censorship: 0
// freedom: MAX
```

```
int main() {
  while (true) {
    think();
    create();
    be_free();
  }
}
```

Complete Hardware + Software Guide
From $0 to Production – No Cloud Required

```
# root@phantombyte:~$
./sovereign_ai_stack  --init
[OK] System online.
```

01 // LAPTOP NODE
- Local Inference
- Dev Environment
- Full Control

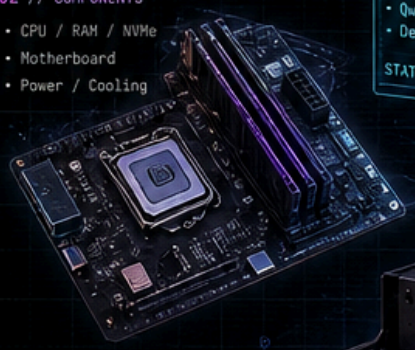
SECURITY BY DESIGN
- Local Only
- Encrypted
- Private
- Untouchable

NO CLOUD.
NO CHAINS.
NO MASTERS.

MODEL LOAD:
- Llama-3-70B
- Qwen2.5-72B
- DeepSeek-R1

STATUS: READY

02 // COMPONENTS
- CPU / RAM / NVMe
- Motherboard
- Power / Cooling



TOKENS/SEC	145.7
CONTEXT	131072
PRECISION	BF16
KV CACHE	98%

04 // SERVER NODE
- 24/7 Uptime
- Model Hosting
- Scalable Compute

03 // DUAL GPU POWER
- Parallel Inference
- Max Throughput
- Unstoppable


```
root@sovereign:~$ neofetch
```

OS: SovereignOS
Kernel: 6.8.0-sov
Uptime: 12 days
Models: Local Only
Shell: zsh
WM: PhantomWM
Terminal: ghostty

Freedom: 100%
Censorship: 0%
Cloud: DISCONNECTED

Run Your Own AI • Own Your Data • Zero Censorship

Llama 3 | Qwen 2.5 | DeepSeek R1

Open Source.
Open Future.
Yours.

PART 1: HARDWARE -- THE SHOPPING LIST

FOUR BUILDS. Pick the one that matches your ambition (and wallet).

TIER 1: THE "$0 REUSE WHAT YOU OWN" BUILD

Use any computer you already have. Seriously. A gaming PC, a MacBook, even an old workstation -- Ollama runs on damn near everything.

Minimum specs to be useful:

- CPU: Any quad-core from the last 8 years (Intel i5-8400 or better)
- RAM: 16 GB DDR4 (8 GB works for 3B models, but 16 is the floor)
- Storage: 50 GB free SSD space (models live on disk)
- GPU: Optional at this tier -- CPU inference works for 7B-8B models

What you can run:

- Qwen 2.5 7B / Llama 3.2 3B / Gemma 3 4B -- CPU only, 5-15 tokens/sec
- DeepSeek-R1 7B/8B -- CPU only, usable for chat + reasoning
- Embedding models (nomic-embed-text, mxbai-embed-large)
- Small code models (Qwen 2.5 Coder 7B)

Total cost: $0. You already own this machine.

TIER 2: THE "BUDGET SOVEREIGNTY" BUILD (~$900-1,200)

Purpose-built for local AI. This is the sweet spot -- it runs 8B-14B models comfortably and can stretch to 32B quantized models.

CPU: AMD Ryzen 5 7600X (6-core, 5.3 GHz, AM5)

- $200 | Why: AM5 platform gives you upgrade path. Intel i5-13600K is the alternative at ~$280 if you prefer Team Blue.

GPU: NVIDIA RTX 4060 Ti 16 GB VRAM

- $430-480 | Why: 16 GB VRAM is the magic number. It fits 13B-14B models fully in GPU memory at 4-bit quantization. The 8 GB version is $100 cheaper but limits you to 7B models on GPU.
- Alternative: Used RTX 3090 24 GB (~$600-700 used) if you can find one. The extra VRAM opens up 32B models. This is the best value move if you're comfortable with the used market.

Motherboard: B650 chipset (AM5) -- ASRock B650M Pro RS or MSI B650 Tomahawk

- $140-180 | Why: PCIe 5.0 ready, 4 RAM slots for future expansion.

RAM: 32 GB (2x16 GB) DDR5-6000 CL30

- $95-110 | Why: 32 GB is the floor for serious local AI. When a model doesn't fit in VRAM, it spills to system RAM. DDR5-6000 keeps that spillover fast. Brand doesn't matter much -- G.Skill, Corsair, TeamGroup.

Storage: 1 TB NVMe SSD (Gen 4)

- $60-75 | Why: Model files are huge. Llama 3.1 70B 4-bit = 40 GB. A few models + embeddings + datasets and 500 GB vanishes. Samsung 990 EVO or WD Black SN770. Gen 4 is worth the extra $10 over Gen 3.

PSU: 750W Gold (Corsair RM750e or Thermaltake GF1)

- $90-100 | Why: Headroom for a second GPU later. Gold efficiency.

Case: Any ATX mid-tower with decent airflow ($60-80)

- Fractal Pop Air, Corsair 4000D, or NZXT H5 Flow.

Cooler: Thermalright Peerless Assassin 120 SE ($35)

- Beats coolers that cost 3x. Air cooling is fine for a 105W CPU.

Total: ~$930-1,160

What you can run on this build:

- Llama 3.1 8B -- GPU only, 80+ tokens/sec
- Qwen 2.5 14B -- GPU only at 4-bit, 50+ tokens/sec
- DeepSeek-R1 14B -- GPU only, reasoning runs
- Gemma 3 12B -- Vision + text on GPU
- Qwen 2.5 Coder 14B -- Full GPU code generation
- Llama 3.1 70B 4-bit -- GPU + CPU offload, 5-8 tokens/sec (usable)
- Any embedding model -- instant

TIER 3: THE "ENTHUSIAST" BUILD (~$2,500-3,500)

The build that makes cloud subscriptions look stupid. Runs 32B-70B models at interactive speeds. Dual GPU ready.

CPU: AMD Ryzen 9 7950X (16-core, 5.7 GHz) -- $520

Or: Intel i9-14900K (24-core) -- $550

GPU (pick one path):

PATH A -- Single GPU:

NVIDIA RTX 4090 24 GB -- $1,600-1,800

Best single consumer GPU. 24 GB runs 34B at 4-bit entirely.

PATH B -- Dual GPU (better value):

2x Used RTX 3090 24 GB -- $1,200-1,400 total

48 GB combined VRAM. Runs 70B models comfortably. This is the enthusiast sweet spot. Requires a bigger PSU and case.

Motherboard: X670E chipset (dual GPU PCIe lanes)

- ASRock X670E Steel Legend or MSI X670E Tomahawk -- $260-300
- Why: You need the PCIe lanes for dual GPU.

RAM: 64 GB (2x32 GB) DDR5-6000 CL30 -- $180-210

- 64 GB means you never worry about spillover. Run 70B models with some GPU offload, keep the rest in fast system RAM.

Storage: 2 TB NVMe SSD Gen 4 -- $110-130

- Samsung 990 Pro or WD Black SN850X

PSU: 1200W Platinum (Seasonic Prime or Corsair HX1200) -- $230-280

- Dual 3090s pull 700W+ under load. Don't cheap out here.

Case: Fractal Meshify 2 XL or Lian Li O11 Dynamic -- $150-180

Cooler: Arctic Liquid Freezer III 360mm AIO -- $120

Total: ~$2,500-3,500 (depending on GPU path)

What you can run:

- Qwen 3 32B -- GPU only at 4-bit, 40+ tokens/sec
- Llama 3.1 70B 4-bit -- Full GPU with dual 3090s, 20+ tokens/sec
- DeepSeek-R1 70B -- Dual GPU, full reasoning at interactive speed
- Gemma 4 26B -- Single GPU, vision + text
- Qwen 2.5 Coder 32B -- Production-grade code gen on GPU
- GPT-OSS 20B -- Full GPU, OpenAI open-weight

TIER 4: THE "F*CK THE CLOUD" WORKSTATION (~$6,000-10,000)

This is the build you get when you've already saved $900/month
on cloud bills and you're reinvesting. Runs 70B-120B models.
Multi-GPU, server-grade optional.

CPU: AMD Threadripper 7960X (24-core, PCIe 5.0, 48 lanes) -- $1,500
Or: AMD Ryzen 9 9950X (16-core, cheaper platform)
Threadripper if you need PCIe lanes for 3-4 GPUs. 9950X otherwise.

GPU: 2x-4x RTX 4090 24 GB or RTX 5090 (when available)

- 2x 4090: 48 GB VRAM -- $3,200-3,600
- 4x 4090: 96 GB VRAM -- $6,400-7,200
- Alternatively: Used A6000 48 GB workstation cards ($2,500 each used)

for denser VRAM in fewer slots.

Motherboard: TRX50 (Threadripper) or X870E (AM5)

- TRX50: ASRock TRX50 WS -- $900
- X870E: ASRock X870E Taichi -- $450

RAM: 128 GB DDR5 (4x32 GB) -- $350-400
Or: 192 GB (4x48 GB) if you want to run MoE models.

Storage: 4 TB NVMe Gen 4 (primary) + 8 TB HDD (model archive)

- SSD: $250-300 | HDD: $140

PSU: 1600W+ (dual PSU or server PSU)

- Super Flower Leadex 1600W or dual Corsair HX1500i

Case: Server chassis or open-air mining frame

- Mining frames are ugly but practical for 4 GPUs: $80
- Phanteks Enthoo Pro 2 Server Edition: $180

Cooling: GPU water blocks + custom loop or just run fans loud

- Custom loop: +$500-800. Air cooling 4 GPUs: undervolt them.

Total: $6,000-10,000+

What you can run:

- Llama 3.1 405B 4-bit (if you have enough VRAM pooled)
- DeepSeek-R1 671B (MoE -- only active params loaded)
- Qwen 3.5 122B -- Multi-GPU, near frontier performance
- GPT-OSS 120B -- Full multi-GPU
- Anything on Ollama's library -- you've got the VRAM
- Fine-tune 7B-14B models locally with QLoRA
- Run multiple models simultaneously (chat + embedding + code gen)

PART 2: THE SOFTWARE STACK

Layer by layer. Install in this order.

LAYER 1: OPERATING SYSTEM

- Ubuntu 24.04 LTS (recommended) -- Best CUDA support, headless-friendly
- Windows 11 + WSL2 -- Works, but Docker and GPU passthrough add friction
- macOS (Apple Silicon) -- Ollama runs native on Metal. No NVIDIA CUDA

but Apple's unified memory is excellent for inference. Skip if you
want multi-GPU or training.

Quick start (Ubuntu):

```
sudo apt update && sudo apt upgrade -y
sudo apt install build-essential curl git htop nvtop -y
```

LAYER 2: NVIDIA DRIVERS + CUDA (skip on Mac)

```
# Ubuntu 24.04 -- Install NVIDIA driver
sudo apt install nvidia-driver-550 -y
sudo reboot
# Verify after reboot
nvidia-smi
# Should show: Driver Version 550.x, CUDA Version 12.4
# Install CUDA toolkit (for custom builds, not strictly required)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install cuda-toolkit-12-4 -y
```

LAYER 3: OLLAMA -- THE MODEL SERVER

```
# Linux / WSL2
curl -fsSL https://ollama.com/install.sh | sh
# macOS -- Download from https://ollama.com/download
# Verify
ollama --version
ollama serve # Starts the server (runs on localhost:11434)
# Pull your first models (start with these)
ollama pull qwen3:14b # Best all-around for 16GB VRAM
ollama pull llama3.1:8b # Reliable workhorse
ollama pull nomic-embed-text # Embeddings for RAG
ollama pull qwen2.5-coder:14b # Code generation
ollama pull deepseek-r1:14b # Reasoning tasks
# Key environment variables (add to ~/.bashrc or ~/.zshrc)
export OLLAMA_HOST=0.0.0.0 # Expose on LAN (optional)
export OLLAMA_NUM_PARALLEL=4 # Concurrent requests
export OLLAMA_KEEP_ALIVE=24h # Keep models in memory
export OLLAMA_MAX_LOADED_MODELS=4 # Max simultaneous models
# For GPU offloading on large models (e.g., 70B on 16GB VRAM)
# Ollama auto-offloads. Set num_gpu layers manually if needed:
# ollama run llama3.1:70b
# Then /set parameter num_gpu 40 (adjust based on your VRAM)
```

LAYER 4: OPEN WEBUI -- THE CHAT INTERFACE

```
# Docker method (recommended -- clean isolation)
docker run -d \
-p 3000:8080 \
--add-host=host.docker.internal:host-gateway \
-v open-webui:/app/backend/data \
--name open-webui \
```

--restart always \

ghcr.io/open-webui/open-webui:main

Access at http://localhost:3000

First run: create admin account

Settings → Connections → Ollama API: http://host.docker.internal:11434

Features you get:

- ChatGPT-style interface for local models
- RAG (upload docs, chat with them)
- Web search integration
- Model management UI
- Multi-user support (give family/team access)
- Mobile-friendly PWA

Alternative: Open WebUI without Docker

pip install open-webui

open-webui serve

LAYER 5: OPENCLAW -- AGENT ORCHESTRATION

Install globally via npm

npm install -g openclaw

Initialize

openclaw init

This creates ~/.openclaw/ config directory

Configure for local Ollama (edit ~/.openclaw/openclaw.json)

Sample config snippet:

{

"models": {

"primary": "ollama/qwen3:14b",

"fast": "ollama/llama3.1:8b",

"reasoning": "ollama/deepseek-r1:14b"

},

"tools": {

"browser": true,

"exec": true,

"memory": true

}

}

Start OpenClaw Gateway

openclaw gateway start

Key capabilities with local models:

- Multi-agent swarms running on your hardware
- Cron jobs for automated tasks (news scraping, monitoring)
- Memory persistence across sessions
- Browser automation for web research
- Telegram/WhatsApp integration

CRITICAL: OpenClaw Gateway + Ollama together = a self-hosted AI

assistant that rivals ChatGPT but runs entirely on your machine.
Your data never leaves your network.

LAYER 6: HERMES AGENT + CUSTOM AGENTS

Hermes Agent is a general-purpose agent framework designed for local model execution. It's what you use when OpenClaw's built-in agent patterns aren't enough.

```
# Clone and install
git clone https://github.com/NousResearch/Hermes-Function-Calling.git
cd Hermes-Function-Calling
pip install -r requirements.txt
# Or use the Ollama-native approach with Hermes models
ollama pull hermes3:8b # Function-calling model
ollama pull command-r-plus:104b # Cohere's agentic model
```

Agent architecture pattern (local stack):

Ollama (model serving)
↓
OpenClaw Gateway (orchestration + tools)
↓
Hermes Agent / Custom Python agents (specialized logic)
↓
Open WebUI (user interface)

Example: Local RAG agent that searches your documents

- 1. Ollama serves nomic-embed-text + qwen3:14b
- 2. OpenClaw handles document ingestion + search trigger
- 3. Custom Python agent does chunking + vector search
- 4. Results surface in Open WebUI chat

LAYER 7: OPTIONAL -- BUT HIGHLY RECOMMENDED

A) n8n -- Workflow Automation

```
docker run -d --name n8n -p 5678:5678 \
-v n8n_data:/home/node/.n8n \
--restart always \
n8nio/n8n
# Access at http://localhost:5678
# Connect to Ollama via HTTP Request nodes → localhost:11434
# Automate: content generation, email processing, data pipelines
```

B) Continue.dev or Cline -- IDE Integration

```
# VS Code Extension: Continue
# Configure to use Ollama:
{
"models": [{
"title": "Qwen 14B",
"provider": "ollama",
"model": "qwen3:14b"
```

```
}]
}
# Now you have local AI code completion + chat in VS Code
C) Anything LLM -- Document RAG Platform
docker run -d -p 3001:3001 \
-v anythingllm:/app/server/storage \
mintplexlabs/anythingllm
# Full RAG pipeline: ingest PDFs, websites, codebases
# Query with your local Ollama models
D) pgvector -- Vector Database for RAG
docker run -d --name pgvector -p 5432:5432 \
-e POSTGRES_PASSWORD=yourpassword \
-v pgvector_data:/var/lib/postgresql/data \
pgvector/pgvector:pg16
# Use with LangChain or LlamaIndex for serious RAG pipelines
E) Docker Compose -- The One-File Stack
# Save this as docker-compose.yml and run `docker compose up -d`
version: '3.8'
services:
ollama:
image: ollama/ollama:latest
ports: ["11434:11434"]
volumes:
  • ollama_data:/root/.ollama
deploy:
resources:
reservations:
devices:
  • driver: nvidia
count: all
capabilities: [gpu]
open-webui:
image: ghcr.io/open-webui/open-webui:main
ports: ["3000:8080"]
volumes:
  • webui_data:/app/backend/data
extra_hosts:
  • "host.docker.internal:host-gateway"
restart: always
n8n:
image: n8nio/n8n
ports: ["5678:5678"]
volumes:
  • n8n_data:/home/node/.n8n
restart: always
volumes:
```

ollama_data:

webui_data:

n8n_data:

One command. Your entire stack is running.

PART 3: THE CONFIGURATION CHECKLIST

Follow this in order. Check off each box. When you're done, you have a production-ready sovereign AI stack.

PHASE 1: BARE METAL (30-60 minutes)

[ ] Assemble hardware (or verify existing machine meets specs)
[ ] Install Ubuntu 24.04 LTS (or verify existing OS)
[ ] Run `sudo apt update && sudo apt upgrade -y`
[ ] Install NVIDIA drivers: `sudo apt install nvidia-driver-550 -y`
[ ] Reboot, verify with `nvidia-smi`
[ ] Install Docker: `sudo apt install docker.io -y`
[ ] Add user to docker group: `sudo usermod -aG docker $USER`
[ ] Log out and back in (docker group takes effect)

PHASE 2: CORE SERVICES (15-30 minutes)

[ ] Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
[ ] Pull base models:

- `ollama pull qwen3:14b`
- `ollama pull llama3.1:8b`
- `ollama pull nomic-embed-text`

[ ] Verify: `curl http://localhost:11434/api/tags` (lists models)
[ ] Set environment variables in ~/.bashrc (OLLAMA_HOST, KEEP_ALIVE, etc.)
[ ] Start Ollama serve (it auto-starts via systemd, but verify)
[ ] Install Open WebUI via Docker
[ ] Access http://localhost:3000, create admin account
[ ] Connect Open WebUI to Ollama (Settings → Connections)

PHASE 3: AGENT INFRASTRUCTURE (15-20 minutes)

[ ] Install Node.js 22+: `curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -`
`sudo apt install nodejs -y`
[ ] Install OpenClaw: `npm install -g openclaw`
[ ] Run `openclaw init`
[ ] Edit ~/.openclaw/openclaw.json -- set models to local Ollama
[ ] Run `openclaw gateway start`
[ ] Verify gateway is running: `openclaw gateway status`
[ ] Configure Telegram bot (optional) for mobile access
[ ] Install Python agent dependencies:
`pip install openai httpx numpy` (Ollama-compatible client)

PHASE 4: HARDENING & OPTIMIZATION (10-15 minutes)

[ ] Firewall: `sudo ufw enable`
`sudo ufw allow 22/tcp` (SSH)
`sudo ufw allow 3000/tcp` (Open WebUI, optionally restrict to LAN)
`sudo ufw allow 11434/tcp` (Ollama -- restrict to localhost if possible)

[ ] Limit Ollama to localhost (edit systemd service):
`sudo systemctl edit ollama.service`
Add: Environment="OLLAMA_HOST=127.0.0.1"
[ ] Set up nginx reverse proxy for HTTPS (optional, for remote access):
`sudo apt install nginx certbot python3-certbot-nginx -y`
[ ] Configure GPU power limits for 24/7 operation:
`sudo nvidia-smi -pl 250` (lowers power limit, reduces heat/noise)
[ ] Set up model auto-loading on boot:
Create systemd service that runs `ollama run qwen3:14b "ping"` on startup

PHASE 5: VALIDATION (5 minutes)

[ ] Test chat: Open http://localhost:3000, start a new chat, ask a question
[ ] Test code gen: "Write a Python function that calculates SHA-256 hashes"
[ ] Test RAG: Upload a PDF in Open WebUI, ask questions about its content
[ ] Test API: `curl http://localhost:11434/api/generate -d '{"model":"qwen3:14b","prompt":"Hello"}'`
[ ] Test OpenClaw: Send a message via Telegram (if configured)
[ ] Check VRAM usage: `nvidia-smi` -- should show model loaded
[ ] Check performance: Run `ollama run qwen3:14b --verbose` and note tokens/sec
POST-SETUP: MODEL EXPANSION PLAN

Once your base stack works, pull these in order:

Week 1 (Core Models):

[ ] ollama pull deepseek-r1:14b (Reasoning)
[ ] ollama pull qwen2.5-coder:14b (Code generation)
[ ] ollama pull gemma3:12b (Vision + fast chat)

Week 2 (Specialized):

[ ] ollama pull llama3.1:70b (Heavy lifting, if VRAM allows)
[ ] ollama pull mxbai-embed-large (Better embeddings for RAG)
[ ] ollama pull mistral:7b (Fast, light alternative)

Week 3 (Frontier):

[ ] ollama pull qwen3.5:27b (Near-frontier performance)
[ ] ollama pull deepseek-r1:32b (Deep reasoning on bigger hardware)
[ ] ollama pull gemma4:26b (Google's latest, if supported)

PART 4: REAL NUMBERS -- CLOUD VS. LOCAL

Comparison based on a developer running AI daily:

CLOUD STACK (monthly):

- ChatGPT Pro: $200
- Claude Pro: $20
- API usage (OpenAI): $150-300 (agent runs, embeddings, testing)
- Cursor Pro: $20
- Perplexity Pro: $20
- Misc (Pinecone, etc): $50-100

Monthly total: $460-660+

LOCAL STACK (one-time, amortize over 12 months):

Tier 2 hardware: $1,100 (one-time → $92/month over 12 months)

Electricity (24/7): $15-25/month (GPU idle ~50W, load ~200W)

Models: $0

API calls: $0

No rate limits: Priceless

Monthly equivalent: ~$110-120

Annual savings: $4,200-6,500. The hardware pays for itself in 2-3 months.

And you get:

- Zero data leaving your network
- No censorship or content filtering
- No rate limits -- run agents 24/7
- No API key management
- No vendor lock-in
- Models keep working even if OpenAI goes down
- Fine-tuning your own models on your own data

PART 5: TROUBLESHOOTING QUICK REFERENCE

PROBLEM: "Ollama says CUDA error / no GPU detected"
FIX: `nvidia-smi` -- if that works, restart Ollama:

`sudo systemctl restart ollama`

If nvidia-smi fails: reinstall drivers.

PROBLEM: "Model loads but runs on CPU only (slow)"
FIX: Check `ollama ps` while model is running -- shows GPU vs CPU layers.

Small models that fit in VRAM should show 100% GPU.

If GPU layers = 0: check CUDA toolkit is installed.

If partial GPU: normal for large models. Adjust with /set parameter num_gpu.

PROBLEM: "Out of memory errors"
FIX: Pull a smaller quant: `ollama pull qwen3:14b-q4_K_M`

Or close other models: `ollama stop `

Or reduce context window: /set parameter num_ctx 4096

PROBLEM: "Open WebUI can't connect to Ollama"
FIX: In Docker: use http://host.docker.internal:11434

Not in Docker: use http://localhost:11434

Check: `curl http://localhost:11434/api/tags` returns JSON

PROBLEM: "Models are slow (low tokens/sec)"
FIX: Check power management: `nvidia-smi -q -d POWER`

GPU might be throttling. Set: `sudo nvidia-smi -pl 300`

Check temperatures: `nvidia-smi -q -d TEMPERATURE`

Above 85C: improve case airflow, undervolt GPU.

PROBLEM: "Disk space disappearing"
FIX: Ollama stores models in ~/.ollama/models/

`ollama list` shows all models and sizes.

Remove unused: `ollama rm `

Check Docker volumes: `docker system df`

PART 6: THE "NEXT LEVEL" -- WHAT TO BUILD AFTER SETUP

Once your stack is running, here's what you can build on it:

- 1. Personal AI Assistant (OpenClaw + Ollama)
- Telegram bot that knows your calendar, email, files
- Runs on your hardware, sees everything you allow
- 2. Code Review Agent (Qwen 2.5 Coder + git hooks)
- Auto-review every commit against your codebase
- Catches bugs before they hit production
- 3. Content Pipeline (OpenClaw swarm + Ollama)
- Multi-agent research → drafting → editing pipeline
- Generates articles, social posts, newsletters
- 4. RAG Knowledge Base (nomic-embed + pgvector + AnythingLLM)
- Ingest your company docs, wikis, codebases
- Chat with your entire knowledge base
- 5. Security Monitoring Agent
- Scan logs, detect anomalies, alert you
- Runs 24/7 on spare GPU cycles
- 6. Fine-Tuning Rig (QLoRA on your hardware)
- Fine-tune 7B-14B models on your own writing style
- Custom models for your specific domain

RESOURCES & LINKS

Ollama: https://ollama.com

Ollama Models: https://ollama.com/library

Open WebUI: https://github.com/open-webui/open-webui

OpenClaw: https://docs.openclaw.ai

n8n: https://n8n.io

pgvector: https://github.com/pgvector/pgvector

Continue.dev: https://continue.dev

AnythingLLM: https://anythingllm.com

Hermes Agent: https://github.com/NousResearch/Hermes-Function-Calling

PhantomByte Articles (related reading):

- "I Cut $900/Month From My Cloud Bill By Going Local"
- "Sovereign AI: Why Your Models Should Run On Your Hardware"
- articles.phantom-byte.com

END OF BLUEPRINT

Built by PhantomByte -- Code from the shadows.

Questions? hello@phantom-byte.com

Ready to go sovereign?

More guides, tools, and tutorials at:

articles.phantom-byte.com

Built by PhantomByte -- Code from the shadows.